NITSHACKS 9.0

IDEA SUBMISSION

NITSHACKS 9.0

PROBLEM STATEMENT

J.A.R.V.I.S. - Autonomous incident investigation, root-cause analysis, and self-healing for distributed systems

THEME

AI-Powered Incident Intelligence & Autonomous SRE Operations

TEAM NAME (REGISTERED ON PORTAL)

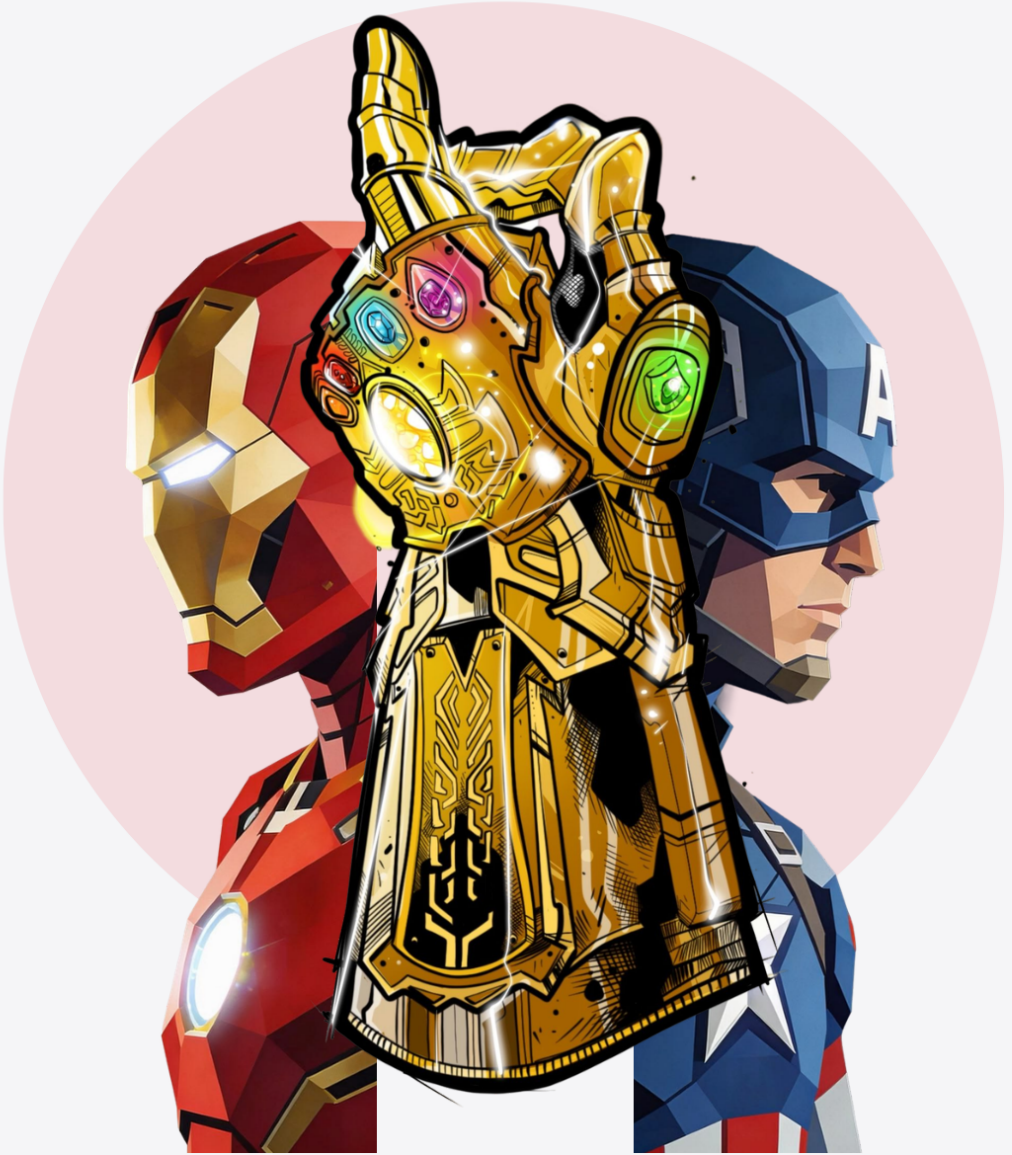
AquaNox

f Incident

Artificial Intelligence, Intelligent Systems & Distributed Systems

TEAM NAME (REGISTERED ON PORTAL)

AquaNox



01 / PROPOSED SOLUTION

J.A.R.V.I.S - SRE Platform

J.A.R.V.I.S. INGESTS □ CORRELATES □ DIAGNOSES □ ANALYZES IMPACT □ REMEDIATES □ VERIFIES

Implemented solution

- REST events, AlertManager webhooks, uploaded LOG/TXT/JSON/JSONL files, and simulated telemetry.
- A unified event schema stores timestamps, severity, service, trace IDs, and metadata.
- Correlation uses time windows, topology distance, shared dependencies, and causal signatures.
- RCA records evidence, confidence, rejected hypotheses, affected services, and business impact.

Why it is innovative

- One master incident instead of duplicate alerts across microservices.
- Governed runbooks execute only within risk policy and approval boundaries.
- Synthetic probes verify recovery before the incident is closed.
- The dashboard exposes live telemetry, timeline, topology, events, and post-mortem export.

ıected services. Engineers
tand impact, and recover the



t probable cause of system

2. **Intelligent Event Correlation**: Connects related alerts across multiple microservices into a single incident, reducing duplicate alerts and alert fatigue.
3. **Automated Impact Analysis & Remediation**: Identifies affected services, visualizes dependencies, and recommends or performs safe recovery actions.
4. **Self-Healing with Recovery Verification**: Verifies system health after remediation to confirm successful recovery and reduce Mean Time to Recovery (MTTR).

NITSHACKS 9.0

02 / BUILD PLAN

TECHNICAL APPROACH

Technologies to be use

NEXT.JS

FastAP

LangGra

docker

PostgreSQL

IMPLEMENTED WORKFLOW

- FastAPI event, batch, webhook, and log-upload APIs.
- SQLAlchemy storage for incidents, events, logs, and impact metrics.
- Correlation engine uses a 120-second window plus service topology.
- LangGraph/LangChain agent workflow with deterministic fallbacks.
- Next.js dashboard with live WebSocket telemetry and REST fallback.
- Timeline, correlated events, benchmarks, and Markdown post-mortem export.

CHALLENGE SCENARIO

- Payment cascade across user, order, payment, and database services.
- One master incident from three different event sources.
- Impact metrics: failed logins, failed transactions, and revenue risk.
- Safe remediation: restart, pool recovery, stuck-job cleanup, rollback.
- Synthetic probes confirm recovery before closure.

licroservices + uploaded logs
ateway · User · Order · Payment · Notification

rometheus + AlertManager
etrics collection and alerts

vent schema + correlation
me window · dependencies · shared service

LangGraph agent
Root cause · impact · governance

Remediation + verification
Restart · rollback · scale · health checks

6 The Governance Agent checks the fix, then a safe action runs.

7 Recovery is verified with health checks and metrics.

8 Dashboard shows incident status, timeline, dependency graph and post-mortem.

03 / REALITY CHECK

FEASIBILITY AND VIABILITY

- Open-source stack with local AI option.
- Mock services and reproducible test logs.
- SQLite for demos; PostgreSQL-ready persistence.
- Modular API boundaries for future observability integrations.

- Noisy or duplicate alerts.
- Incorrect model hypotheses.
- Risky automated actions.
- False recovery signals.
- Unavailable local model.

- Correlation thresholds and trace/topology evidence.
- Deterministic RCA fallback rules.
- Risk classification and approval gate.
- Health probes and bounded runbooks.
- Audit logs and generated post-mortems.

persistent deployment.

- Modular services make future integration with real infrastructure possible.

Local model performance limitations

Data loss or service failure

Mitigation

Correlation rules, grouping, and time-window analysis

Use metric evidence, dependency data, and confidence scores

Policy checks, approval for risky actions, and action limits

Verify health checks, error rates, latency, and service availability

Keep workflows lightweight and use deterministic checks alongside AI

Database backups, logs, and controlled rollback procedures

04 / WHY IT MATTERS

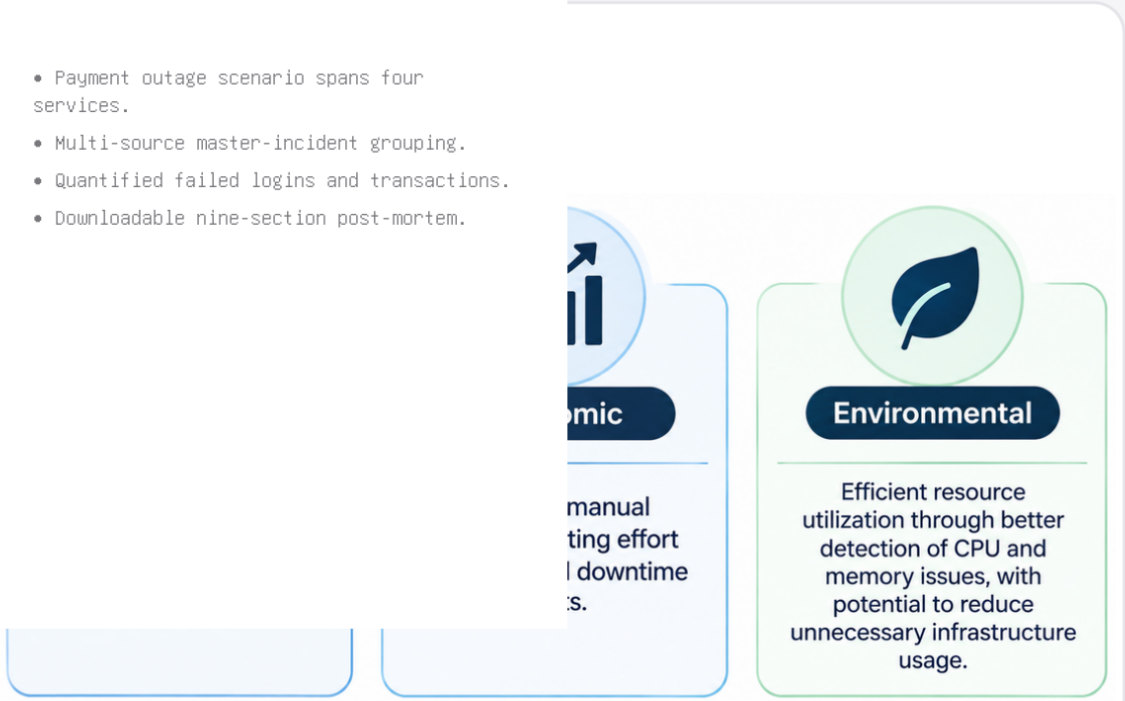
IMPACT AND BENEFITS

- SRE and DevOps teams.
- Cloud and infrastructure engineers.
- Software operations teams.
- Incident response and IT support.

- Faster investigation and recovery.
- Reduced duplicate-alert fatigue.
- Clear root cause and blast-radius visibility.
- Consistent timelines and reports.

- Payment outage scenario spans four services.
- Multi-source master-incident grouping.
- Quantified failed logins and transactions.
- Downloadable nine-section post-mortem.

- Reduced alert fatigue through event correlation.
- Better visibility into root causes and affected services.
- Consistent incident timelines and postmortem reports.
- Safer, more structured remediation workflows.



05 / EVIDENCE

RESEARCH AND REFERENCES

PROJECT EVIDENCE

- GitHub repository: Rahul3106/Jarvis
- README architecture and API reference
- test_logs/ multi-source fixtures
- backend/tests/ phase-based verification suite
- Interactive Next.js dashboard and FastAPI backend

ENGINEERING FOUNDATION

- Google SRE principles for monitoring and incident response
- Graph-based dependency analysis
- Human-in-the-loop operational governance
- Evidence-driven root-cause and recovery verification

Distributed Systems (2020)
igate and debug complex
ed systems.

istributed Systems
ty, symptoms versus root

causes, and incident detection.
Click here

LLM
Analysis and Incident Response in Microservices (2026)
Research on combining LLM agents with service dependency graphs for incident investigation and response.
Click here